



MODUL PRAKTIKUM

Sistem Operasi

IF25-12007 | Genap 2025/2026

Mata Kuliah Sistem Operasi (IF25-12007)

Semester Genap 2025/2026

Institusi Institut Teknologi Sumatera

TIM PENGAJAR & ASISTEN

Dosen Pengampu

Martin Clinton Tosima Manullang, S.T., M.T., Ph.D.

Ilham Firman Ashari, S.Kom., M.T

I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.

Eko Dwi Nugroho, S.Kom., M.Cs.

Asisten Praktikum

Ahmat Prayoga Sembiring	123140053
Andryano Shevchenko Limbong	123140205
Aziz Kurniawan	122140097
Dito Rifki Irawan	122140153
Ebentua Philippus Limbong	123140086
Elsa Elisa Yohana Sianturi	122140135
Fathan Andi Kartagama	122140055
Miftahul Khoiriyah	123140064
Muhammad Bintang Al-Fasya	123140098
Muhammad Fadhil Zurani	122140146
Reza Chairul Manam	120140086
Stevanus Cahya Anggara	123140038
Varasina Farmadani	123140107
Zakhi Algifari	122140198

Contents

1	Penjadwalan Proses: Round-Robin dan Priority	4
1.1	Tujuan	4
1.2	Sesi 1 — Mengamati Perilaku Round-Robin	4
1.2.1	Persiapan dan Kompilasi	4
1.2.2	Analisis Perilaku Round-Robin	5
1.3	Sesi 2 — Implementasi Priority Scheduling	6
1.3.1	Analisis Kode yang Akan Dirombak	6
1.3.2	Implementasi <code>next_thread_to_run()</code> (Priority)	6
1.4	Sesi 3 — Kompilasi dan Verifikasi	7
1.5	Eksplorasi Mandiri	9

CHAPTER 1

Penjadwalan Proses: Round-Robin dan Priority

1.1 Tujuan

Kerjakan tugas-tugas berikut secara berurutan mengikuti alur sesi pada Modul 11 dan Modul 12. Sertakan bukti berupa tangkapan layar (*screenshot*) QEMU atau penggalan kode yang relevan pada setiap langkah yang diminta.

1.2 Sesi 1 — Mengamati Perilaku Round-Robin

1.2.1 Persiapan dan Kompilasi

Pastikan implementasi Minggu 4 sudah berfungsi, lalu kompilasi dengan *flag* `PRIORITY_SCHEDULING` diaktifkan.

1. Pastikan Anda sudah berada di dalam lingkungan Docker (jalankan `./run-docker.sh` jika belum) sebelum menjalankan perintah berikut.
2. Jalankan perintah berikut untuk mengompilasi dan menjalankan kernel:

```
1 make clean
2 make DEMO=-DPRIORITY_SCHEDULING
```

```
3 make run
4
```

Kode 1.1: Compile dan jalankan demo Priority Scheduling

Amati keluaran (*output*) yang muncul selama ± 15 detik, lalu ambil tangkapan layar (*screenshot*) pada layar QEMU. Tempel hasil tangkapan layar di sini:

[Tempel screenshot layar QEMU di sini]

Gambar 1.1: Tampilan QEMU dengan scheduler Round-Robin (ketiga thread bergantian tanpa memperhatikan prioritas).

- Perhatikan tiga thread yang dibuat di `kernel.c` dan isilah tabel berikut berdasarkan kode sumber.

Tabel 1.1: Konfigurasi thread pada mode `PRIORITY_SCHEDULING`

Nama	Prioritas	Warna VGA	max_ticks
Low	10	<i>[Isi]</i>	<i>[Isi]</i>
Med	<i>[Isi]</i>	<i>[Isi]</i>	<i>[Isi]</i>
High	<i>[Isi]</i>	<i>[Isi]</i>	<i>[Isi]</i>

1.2.2 Analisis Perilaku Round-Robin

- Perhatikan urutan keluaran thread di layar QEMU. Isilah tabel berikut untuk mengidentifikasi pola urutan eksekusi.
- Apakah thread High (prioritas 50) mendapat perlakuan istimewa dari *scheduler*? Jelaskan mengapa atau mengapa tidak, berdasarkan kode `next_thread_to_run()` versi Round-Robin.

Jawaban:

[Isi jawaban di sini]

Tabel 1.2: Urutan eksekusi thread yang diamati pada Round-Robin

Giliran	Thread	Prioritas	Mengapa dipilih?
1	[Isi]	[Isi]	[Isi: karena masuk antrian pertama]
2	[Isi]	[Isi]	[Isi]
3	[Isi]	[Isi]	[Isi]
4	[Isi]	[Isi]	[Isi: kembali ke depan (FIFO berputar)]

3. Setelah High selesai (tick ke-5), apa yang terjadi pada urutan eksekusi antara Low dan Med? Apakah keduanya tetap bergantian secara adil?

Jawaban:

[Isi jawaban di sini]

1.3 Sesi 2 — Implementasi Priority Scheduling

1.3.1 Analisis Kode yang Akan Dirombak

1. Buka file `src/thread.c` dan perhatikan fungsi `next_thread_to_run()` versi FIFO. Identifikasi baris kode mana yang menyebabkan scheduler mengabaikan prioritas.

Tabel 1.3: Identifikasi kelemahan kode Round-Robin

Pertanyaan	Jawaban
Baris kode mana yang mengambil thread dari antrian?	[Isi: tuliskan baris kode tersebut]
Apakah baris tersebut memeriksa field <code>priority</code> ?	[Isi]
Apa yang menentukan urutan pemilihan thread?	[Isi: urutan masuk antrian / nilai prioritas / ...]

1.3.2 Implementasi `next_thread_to_run()` (Priority)

1. Salin kode `next_thread_to_run()` versi Priority Scheduling yang telah Anda tulis:

```
1 struct thread *next_thread_to_run(void) {
2
```

```
3      /* Tulis implementasi Anda di sini */
4
5  }
6
```

Kode 1.2: Implementasi `next_thread_to_run()` versi Priority oleh mahasiswa

2. Jelaskan secara singkat logika tiga tahap pada implementasi Anda:

- (a)** Tahap 1 (Pemindaian): Bagaimana Anda menelusuri isi antrian untuk mencari thread dengan prioritas tertinggi?

Jawaban:

[Isi jawaban di sini]

- (b)** Tahap 2 (Pengambilan): Mengapa Anda perlu menyimpan pointer ke thread terpilih sebelum melakukan penggeseran?

Jawaban:

[Isi jawaban di sini]

- (c)** Tahap 3 (Penggeseran): Mengapa perlu menggeser elemen setelah posisi terpilih? Apa yang terjadi jika langkah ini dilewati?

Jawaban:

[Isi jawaban di sini]

- 3.** Jika dua thread memiliki prioritas yang sama (misalnya keduanya `PRI_DEFAULT`), thread mana yang akan dipilih oleh implementasi Anda? Apakah ini perilaku yang diinginkan? (Kaitkan jawaban Anda dengan hasil eksperimen pada Eksplorasi 2)

Jawaban:

[Isi jawaban di sini]

1.4 Sesi 3 — Kompilasi dan Verifikasi

- 1.** *Compile* ulang dan jalankan kernel setelah implementasi Priority Scheduling:

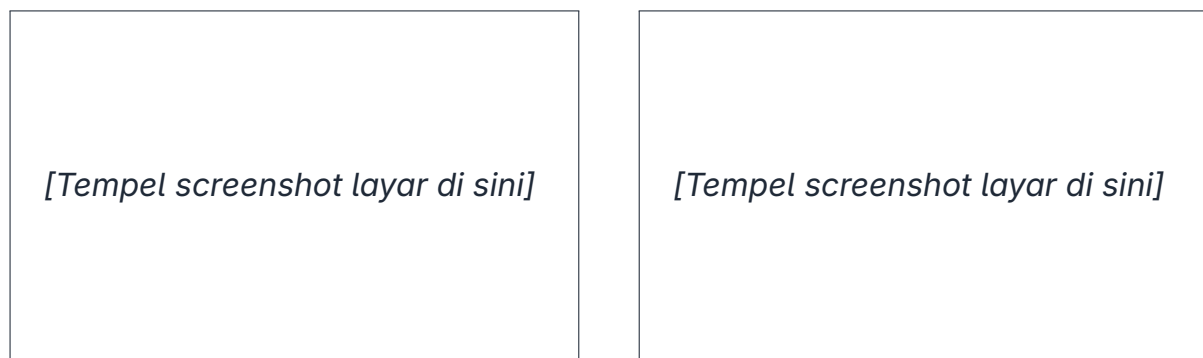
```

1 make clean
2 make DEMO=-DPRIORITY_SCHEDULING
3 make run
4

```

Kode 1.3: Compile dan jalankan dengan Priority Scheduling

Sertakan tangkapan layar (*screenshot*) keluaran (*output*) QEMU **sebelum** dan **sesudah** modifikasi scheduler secara berdampingan.



(a) Sebelum modifikasi (Round-Robin)

(b) Setelah modifikasi (Priority Scheduling)

Gambar 1.2: Perbandingan tampilan QEMU sebelum dan sesudah implementasi Priority Scheduling.

2. Verifikasi urutan keluaran (*output*) yang benar. Isilah tabel berikut berdasarkan pengamatan Anda.

Tabel 1.4: Perbandingan urutan eksekusi sebelum dan sesudah modifikasi

Giliran ke-	Round-Robin (Modul 11)	Priority Scheduling (Modul 12)
1	[Isi: Low / Med / High?]	[Isi]
2	[Isi]	[Isi]
3	[Isi]	[Isi]
4	[Isi]	[Isi]
6 (setelah High exit)	[Isi]	[Isi]

3. Apakah thread Low (prioritas 10) mendapat giliran setelah thread High selesai? Jelaskan mengapa hal tersebut terjadi (kaitkan dengan mekanisme pemindaian antrian pada skenario ini).

Jawaban:

[Isi jawaban di sini]

4. Isilah tabel ringkasan perbandingan kedua algoritma berdasarkan hasil praktikum yang telah dilakukan.

Tabel 1.5: Ringkasan perbandingan Round-Robin vs Priority Scheduling

Aspek	Round-Robin	Priority Scheduling
Prinsip pemilihan	[Isi]	[Isi]
Fairness	[Isi]	[Isi]
Apakah High didahulukan?	[Isi]	[Isi]
Risiko utama	[Isi]	[Isi]

1.5 Eksplorasi Mandiri

1. Eksperimen mengubah nilai prioritas.

Pada `kernel.c`, tukar nilai prioritas thread Low dan High sehingga Low mendapat prioritas 50 dan High mendapat prioritas 10. *Compile* ulang dan jalankan.

```
1 make clean && make DEMO=-DPRIORITY_SCHEDULING && make run
2
```

Kode 1.4: Kompilasi untuk eksperimen tukar prioritas

(a) Sertakan tangkapan layar (*screenshot*) hasil QEMU.

[Tempel screenshot layar di sini]

Gambar 1.3: Tampilan QEMU setelah prioritas Low dan High ditukar.

(b) Thread mana yang sekarang dieksekusi terlebih dahulu? Apakah hasilnya sesuai dengan ekspektasi?

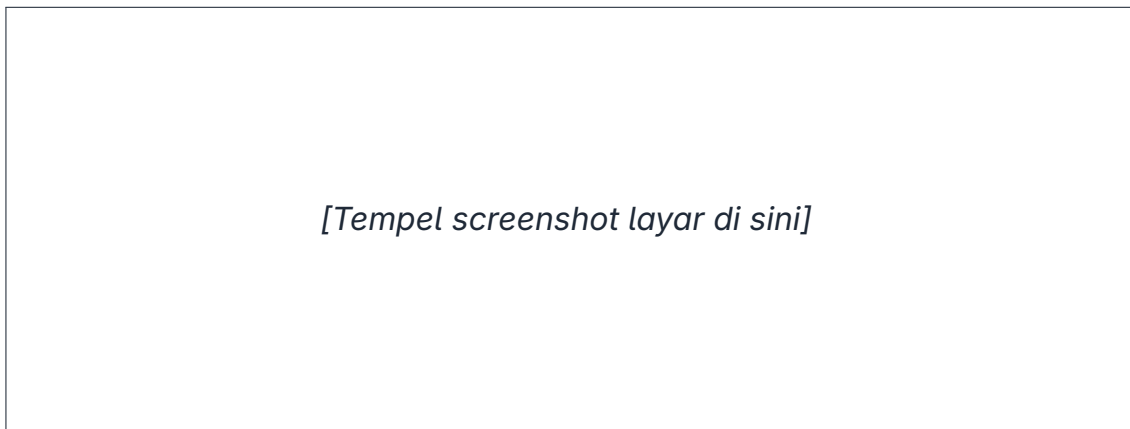
Jawaban:

[Isi jawaban di sini]

2. Simulasi Round-Robin dengan Prioritas.

Ubah parameter prioritas pada ketiga *thread* di *kernel.c* agar semuanya memiliki prioritas yang **sama persis** (misalnya ketiganya diberi `PRI_DEFAULT` atau 31). Pastikan `max_ticks` untuk High tetap 5. *Compile* ulang dan jalankan.

(a) Sertakan tangkapan layar (*screenshot*) hasil QEMU.



Gambar 1.4: Tampilan QEMU saat ketiga thread memiliki prioritas yang sama.

(b) Bagaimana perilaku *scheduler* saat semua antrian memiliki nilai prioritas yang setara? Apakah kembali bertindak seperti *Round-Robin* (FIFO)? Jelaskan alasannya berdasarkan implementasi kode Anda.

Jawaban:

[Isi jawaban di sini]

3. Refleksi: Solusi untuk starvation.

Berdasarkan konsep *aging* yang telah dipelajari pada dasar teori Modul 12:

(a) Jelaskan bagaimana teknik *aging* dapat mengatasi *starvation* yang Anda amati pada eksperimen di atas.

Jawaban:

[Isi jawaban di sini]

(b) Komponen apa yang perlu ditambahkan pada `next_thread_to_run()` agar *aging* dapat bekerja? Jelaskan secara konseptual (tidak perlu menulis kode).

Jawaban:

[Isi jawaban di sini]